

SALESTORM: SOLID PRINCIPLES MAPPING

Team: Binary Minds

1. Purpose

This document maps the five SOLID principles to actual SALESTORM services, classes and interfaces. The goal is clear separation of responsibilities, minimal coupling and an implementation-ready design for the flash-sale workflow.

2. SOLID Summary

Principle	Meaning	SALESTORM Application	Primary Area
SRP	One class/service has one responsibility	Payment, Order, Inventory and Notification responsibilities are separated	Service boundaries
OCP	Open for extension, closed for modification	Add payment/pricing strategies without changing core checkout logic	Payment / Pricing
LSP	Subtypes can replace their abstractions	Card and UPI implementations remain valid PaymentProcessor implementations	Payment
ISP	Use focused interfaces	Small interfaces such as PaymentGateway, InventoryRepository and NotificationSender	Interfaces
DIP	Depend on abstractions, not concrete implementations	Checkout/Payment depend on interfaces and inject provider implementations	Service dependencies

3. Design Principle

```
Customer
|
v
Checkout Service
|
+--> InventoryService
|
+--> PaymentProcessor
|    +--> CardPaymentProcessor
|    +--> UPIPaymentProcessor
|
+--> OrderService
|
+--> NotificationSender
```

The design keeps business logic independent from provider-specific implementations. This supports the hackathon requirement to show clear SOLID application and accommodate a new payment provider, pricing strategy or delivery partner with minimal change.

SOLID MAPPING

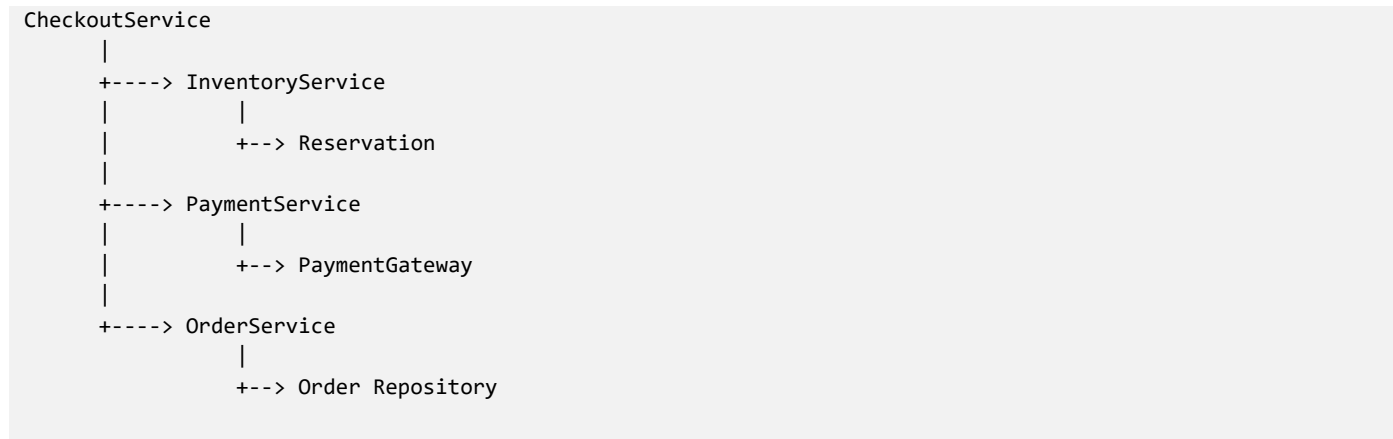
S — Single Responsibility Principle

4. SRP — Single Responsibility Principle

The hackathon expects payment processing, order management and notification responsibilities to be separated. SALESTORM applies SRP by keeping each critical service focused on one business responsibility.

Component	Single Responsibility	What it does NOT own
InventoryService	Inventory availability and reservation lifecycle	Payment processing / order creation
PaymentService	Payment initiation, status and provider interaction	Inventory ownership / shipment
OrderService	Order creation and order state lifecycle	Direct payment-provider logic
NotificationService	Customer notification delivery	Changing inventory or order state
CheckoutService	Coordinates the purchase workflow	Persisting provider-specific payment details

5. SRP in the Purchase Flow



Why this helps: a change to notification delivery should not require changes to payment logic; a new payment provider should not require changes to order-state logic.

6. SRP Benefit

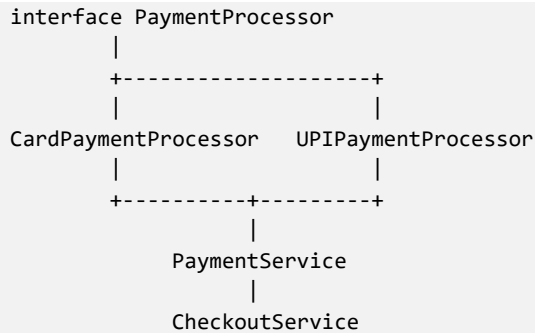
- Reduces coupling between critical purchase components.
- Makes failures easier to isolate and recover.
- Allows services to scale independently.
- Makes classes easier to test and explain during the jury defense.

SOLID MAPPING

O — Open/Closed, L — Liskov Substitution

7. OCP — Open/Closed Principle

SALESTORM should allow new payment providers or pricing strategies to be added without modifying the core checkout workflow.



Extension	Existing Core Logic	Change Required
New payment provider	Checkout + PaymentService	Add a new PaymentProcessor implementation
New pricing strategy	Checkout workflow	Add a PricingStrategy implementation
New delivery partner	Shipment workflow	Add a DeliveryProvider adapter

8. LSP — Liskov Substitution Principle

Any concrete implementation of a payment abstraction must be usable wherever the abstraction is expected, without breaking the PaymentService contract.

```
PaymentProcessor processor

processor = CardPaymentProcessor
processor.process(paymentRequest)

processor = UPIPaymentProcessor
processor.process(paymentRequest)

# PaymentService uses the same abstraction
# regardless of the concrete provider.
```

9. LSP Rules in SALESTORM

- Every PaymentProcessor accepts the defined payment request contract.
- Each implementation returns a valid PaymentResult.
- Failures are represented through the agreed error/result model.
- PaymentService does not need provider-specific conditional logic.

SOLID MAPPING

I — Interface Segregation, D — Dependency Inversion

10. ISP — Interface Segregation Principle

SALESTORM uses focused interfaces so a component depends only on the operations it actually needs.

Focused Interface	Used By	Responsibility
PaymentGateway	PaymentService	Charge, query status and refund
InventoryRepository	InventoryService	Read and update inventory
OrderRepository	OrderService	Persist and retrieve orders
NotificationSender	NotificationService	Send customer notifications
DeliveryProvider	ShipmentService	Create shipment and track delivery

11. DIP — Dependency Inversion Principle

High-level business services depend on abstractions. Concrete infrastructure or third-party providers are supplied through interfaces.

```
CheckoutService
  |
  v
PaymentProcessor (interface)
  |
  +--> CardPaymentProcessor
  +--> UPIPaymentProcessor

PaymentProcessor
  |
  v
PaymentGateway (interface)
  |
  +--> ExternalGatewayAdapter
```

12. Dependency Direction

- Business logic → interfaces
- Interfaces → concrete implementations through dependency injection
- External providers are isolated behind adapters
- Repositories abstract PostgreSQL persistence

13. Why DIP Matters for SALESTORM

If the payment gateway changes, PaymentService does not need to know the gateway's implementation details. A new provider can be introduced behind the existing abstraction. This directly supports the hackathon's requirement for minimal change when adding providers.

SOLID DEFENSE & TRACEABILITY

How the principles connect to SALESTORM

14. End-to-End SOLID Mapping

Principle	SALESTORM Class / Component	Design Decision	Result
SRP	InventoryService	Owns reservation and stock operations	Focused responsibility
SRP	PaymentService	Owns payment workflow	Independent payment changes
OCP	PaymentProcessor	New provider = new implementation	Minimal modification
LSP	CardPaymentProcessor / UPIPaymentProcessor	Both satisfy PaymentProcessor contract	Interchangeable implementations
ISP	PaymentGateway / Repository interfaces	Small, focused contracts	Reduced unnecessary dependency
DIP	CheckoutService → PaymentProcessor	High-level logic uses abstraction	Low coupling

15. Trade-offs

- More interfaces and abstractions increase design complexity.
- Dependency injection requires additional wiring/configuration.
- The benefit is lower coupling, easier testing and safer extension of the system.
- For a flash-sale system, this separation helps isolate changes and failures in payment, inventory and external providers.